

Teoria Współbieżności

Laboratorium 8

Asynchroniczne wykonanie zadań w puli wątków przy użyciu wzorców Executor i Future

Kacper Bieniasz

25 listopada 2024

1 Dane techniczne sprzętu

Obliczenia zostały wykonane na komputerze o następującej specyfikacji:

- Procesor: AMD Ryzen 7 5800U 8 rdzeeni, 16 wątków
- Pamięć RAM: 16 GB DDR4 3200 MHz (2×8GB)
- System operacyjny: Windows 11 Home x64

2 Treść zadania

1. Proszę zaimplementować przy użyciu Executor i Future program wykonujący obliczanie zbioru Mandelbrota w puli wątków. Jako podstawę implementacji proszę wykorzystać kod w Javie (1).
2. Proszę przetestować szybkość działania programu w zależności od implementacji Executora i jego parametrów (np. liczba wątków w puli). Czas obliczeń można zwiększać manipulując parametrami problemu, np. liczbą iteracji (MAX_ITER).

Kod dostarczony z treścią zadania:

```
1 import java.awt.Graphics;
2 import java.awt.image.BufferedImage;
3 import javax.swing.JFrame;
4
5 public class Mandelbrot extends JFrame {
6     private final int MAX_ITER = 570;
7     private final double ZOOM = 150;
8     private BufferedImage I;
9     private double zx, zy, cX, cY, tmp;
10    public Mandelbrot() {
11        super("Mandelbrot Set");
12        setBounds(100, 100, 800, 600);
13        setResizable(false);
14        setDefaultCloseOperation(EXIT_ON_CLOSE);
15        I = new BufferedImage(getWidth(), getHeight(),
16            BufferedImage.TYPE_INT_RGB);
17        for (int y = 0; y < getHeight(); y++) {
18            for (int x = 0; x < getWidth(); x++) {
19                zx = zy = 0;
20                cX = (x - 400) / ZOOM;
21                cY = (y - 300) / ZOOM;
22                int iter = MAX_ITER;
```

```

22         while (zx * zx + zy * zy < 4 && iter > 0) {
23             tmp = zx * zx - zy * zy + cX;
24             zy = 2.0 * zx * zy + cY;
25             zx = tmp;
26             iter--;
27         }
28         I.setRGB(x, y, iter | (iter << 8));
29     }
30 }
31 }
32 @Override
33 public void paint(Graphics g) {
34     g.drawImage(I, 0, 0, this);
35 }
36 public static void main(String[] args) {
37     new Mandelbrot().setVisible(true);
38 }
39 }

```

Kod Java 1: Kod do wykorzystania

3 Implementacja wykorzystująca Executor oraz Future

Porównanie wyników wydajności w zależności od maksymalnej liczby iteracji, liczby wątków oraz użytego Executora realizuje klasa **MandelbrotPerformanceTest**.

Testowane rodzaje Executorów:

- `newSingleThreadExecutor`
- `newFixedThreadPool`
- `newCachedThreadPool`
- `newWorkStealingPool`

Sprawdzana liczba wykorzystanych wątków: {1, 2, 4, 8, 10, 12, 16, 18}

Wartości maksymalnej liczby iteracji w celu testowania wydajności: {500, 1000, 2000, 5000, 10000}

3.1 Atrybuty oraz konstruktor klasy MandelbrotPerformanceTest

Blok (2) zawiera pola odpowiadające za wyświetlenie wyznaczonego zbioru. Nas najbardziej interesuje pole *maxIter* ponieważ zawiera ono maksymalną liczbę iteracji dla danego wykonania programu.

```

1     private final int width = 800;
2     private final int height = 600;
3     private final int maxIter;
4     private final double zoom = 150;
5     private BufferedImage I;
6     public MandelbrotPerformanceTest(int maxIter) {
7         this.maxIter = maxIter;
8         I = new BufferedImage(width, height, BufferedImage.TYPE_INT_RGB);
9     }

```

Kod Java 2: Atrybuty oraz konstruktor klasy **MandelbrotPerformanceTest**

3.2 Metody obliczające wartości zbioru Mandelbrota *compute()* oraz *computePixel()*

Metoda *compute()* jest nadrzędna, wywołuje *computePixel()* ułatwia to czytelność. Zadaniem obu metod jest obliczenie wartości danego piksela w zbiorze Mandelbrota. Kod obu metod znajduje się w bloku (3).

```
1 public void compute(ExecutorService executor) {
2     List<Future<Void>> futures = new ArrayList<>();
3     for (int y = 0; y < height; y++) {
4         int finalY = y;
5         futures.add(executor.submit(() -> {
6             for (int x = 0; x < width; x++) {
7                 computePixel(x, finalY);
8             }
9             return null;
10        }));
11    }
12    for (Future<Void> future : futures) {
13        try {
14            future.get();
15        } catch (Exception e) {
16            e.printStackTrace();
17        }
18    }
19 }
20 private void computePixel(int x, int y) {
21     double zx = 0, zy = 0, cX, cY, tmp;
22     cX = (x - 400) / zoom;
23     cY = (y - 300) / zoom;
24     int iter = maxIter;
25     while (zx * zx + zy * zy < 4 && iter > 0) {
26         tmp = zx * zx - zy * zy + cX;
27         zy = 2.0 * zx * zy + cY;
28         zx = tmp;
29         iter--;
30     }
31     I.setRGB(x, y, iter | (iter << 8));
32 }
```

Kod Java 3: Metody realizujące obliczenia

3.3 Metoda *main()*

Test wydajności w zależności od parametrów wykonuje metoda *main*. W pętlach iteruje po parametrach wykonania programu. Realizując *FixedThreadPool* testuję różną liczbę wątków. Podobnie *executor WorkStealingPool* do którego przekazujemy parametr, który określa maksymalną liczbę wątków aktywnych. *CachedThreadPool* wykorzystuje maksymalną dostępną liczbę wątków, natomiast *SingleThreadExecutor* działa zawsze na jednym wątku. Po wykonaniu zadania przez wątki z danego executora dane wypisywane na konsolę oraz zapisywane w pliku w celu dalszej analizy.

```
1 public static void main(String[] args) {
2     int[] maxIterValues = {500, 1000, 2000, 5000, 10000};
3     int[] threadCounts = {1, 2, 4, 8, 10, 12, 16, 18};
4     String outputFile = "mandelbrot_results.csv";
5     try (FileWriter writer = new FileWriter(outputFile)) {
6         writer.write("MAX_ITER,ExecutorType,ThreadCount,Time(ms)\n");
7
8         for (int maxIter : maxIterValues) {
9             System.out.println("Test dla MAX_ITER = " + maxIter);
10            for (int threads : threadCounts) {
```

```

11         long start = System.currentTimeMillis();
12         ExecutorService executor =
13             Executors.newFixedThreadPool(threads);
14         MandelbrotPerformanceTest mandelbrot = new
15             MandelbrotPerformanceTest(maxIter);
16         mandelbrot.compute(executor);
17         executor.shutdown();
18         long end = System.currentTimeMillis();
19         long elapsedTime = end - start;
20         System.out.println("Liczba wątków (FixedThreadPool): " +
21             threads + ", czas: " + elapsedTime + " ms");
22         writer.write(maxIter + ",FixedThreadPool," + threads +
23             "," + elapsedTime + "\n");
24     }
25
26     long startSingle = System.currentTimeMillis();
27     ExecutorService singleExecutor =
28         Executors.newSingleThreadExecutor();
29     MandelbrotPerformanceTest mandelbrotSingle = new
30         MandelbrotPerformanceTest(maxIter);
31     mandelbrotSingle.compute(singleExecutor);
32     singleExecutor.shutdown();
33     long endSingle = System.currentTimeMillis();
34     long elapsedSingle = endSingle - startSingle;
35     System.out.println("SingleThreadExecutor, czas: " +
36         elapsedSingle + " ms");
37     writer.write(maxIter + ",SingleThreadExecutor,1," +
38         elapsedSingle + "\n");
39
40     long startCached = System.currentTimeMillis();
41     ExecutorService cachedExecutor =
42         Executors.newCachedThreadPool();
43     MandelbrotPerformanceTest mandelbrotCached = new
44         MandelbrotPerformanceTest(maxIter);
45     mandelbrotCached.compute(cachedExecutor);
46     cachedExecutor.shutdown();
47     long endCached = System.currentTimeMillis();
48     long elapsedCached = endCached - startCached;
49     System.out.println("CachedThreadPool, czas: " + elapsedCached
50         + " ms");
51     writer.write(maxIter + ",CachedThreadPool,0," + elapsedCached
52         + "\n");
53
54     for(int parallel: threadCounts) {
55         long startWorkStealing = System.currentTimeMillis();
56         ExecutorService workStealingExecutor =
57             Executors.newWorkStealingPool(parallel);
58         MandelbrotPerformanceTest mandelbrotWorkStealing = new
59             MandelbrotPerformanceTest(maxIter);
60         mandelbrotWorkStealing.compute(workStealingExecutor);
61         workStealingExecutor.shutdown();
62         long endWorkStealing = System.currentTimeMillis();
63         long elapsedWorkStealing = endWorkStealing -
64             startWorkStealing;
65         System.out.println("WorkStealingPool " + parallel + "
66             czas: " + elapsedWorkStealing + " ms");
67         writer.write(maxIter + ",WorkStealingPool," + parallel +
68             "," + elapsedWorkStealing + "\n");
69     }
70     System.out.println();
71 }
72 } catch (IOException e) {

```

```

56         e.printStackTrace();
57     }
58     System.out.println("Wyniki zapisano do pliku: " + outputFile);
59 }

```

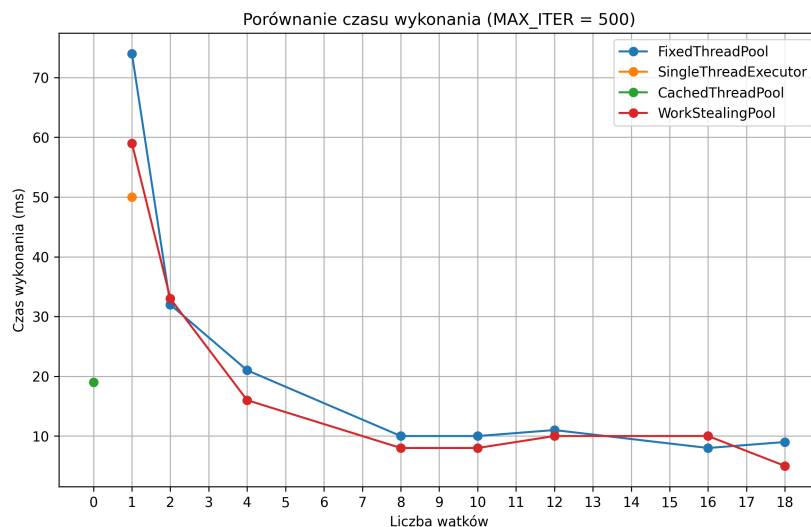
Kod Java 4: Klasa **Proxy**

4 Wyniki i obserwacje

Analiza wyników działania programu postanowiłem porównać ze względu na maksymalną liczbę iteracji. Dla *ChachedThreadPool* jako liczbę wątków ustawiam na 0.

4.1 Wyniki dla $MAX_ITER = 500$

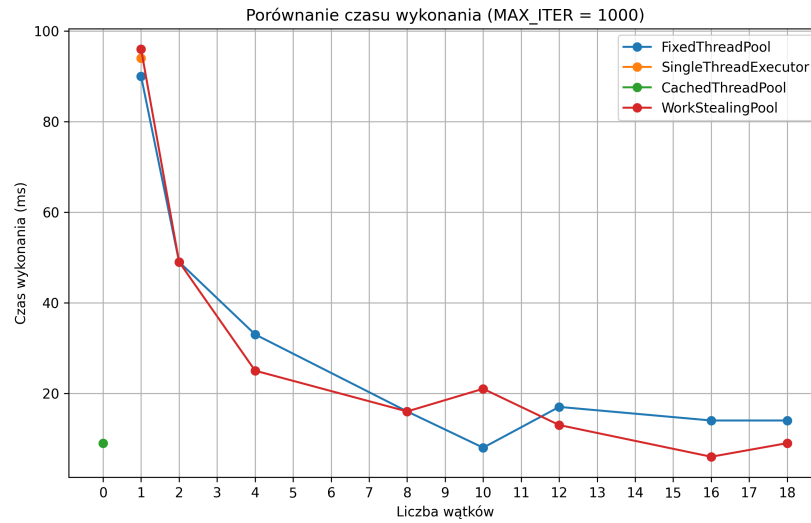
SingleThreadExecutor jest najwolniejszy, co wynika z ograniczenia do jednego wątku, jednak kiedy porównamy go do WorkStealingPool i FixedThreadPool działających na jednym wątku to jest szybszy. WorkStealingPool i FixedThreadPool zaczynają osiągać zbliżoną wydajność od 4 wątków, co sugeruje efektywne wykorzystanie równoległości. CachedThreadPool wykazuje najszybszy czas dla małej liczby zadań, ponieważ od razu tworzy potrzebne wątki.



Rysunek 1: Wykres czasu wykonania obliczeń w zależności od liczby wątków i użytego Executora dla $MAX_ITER = 500$

4.2 Wyniki dla $MAX_ITER = 1000$

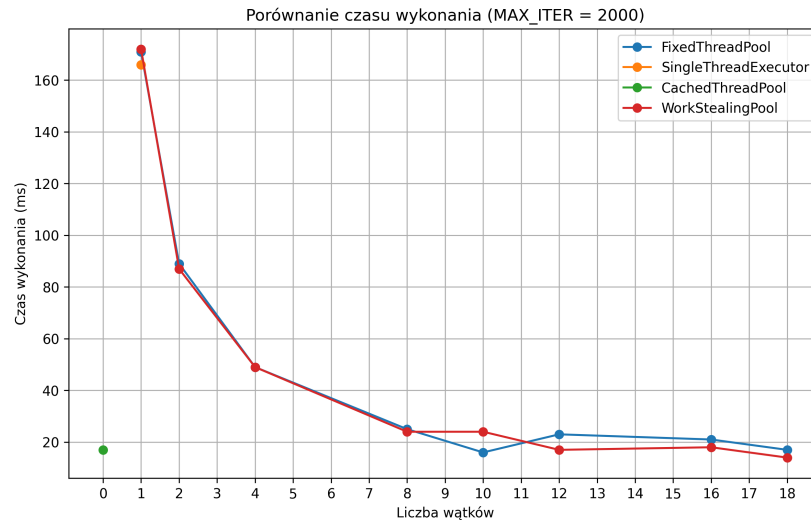
Wraz ze wzrostem liczby iteracji wszystkie implementacje osiągają lepsze wyniki przy większej liczbie wątków. WorkStealingPool ma lekką przewagę nad FixedThreadPool, co może wynikać z efektywnej "kradzieży" zadań w przypadku nierównomiernego obciążenia wątków. CachedThreadPool wciąż działa bardzo szybko, choć różnica w porównaniu do innych puli wątków maleje.



Rysunek 2: Wykres czasu wykonania obliczeń w zależności od liczby wątków i użytego Executora dla $MAX_ITER = 1000$

4.3 Wyniki dla $MAX_ITER = 2000$

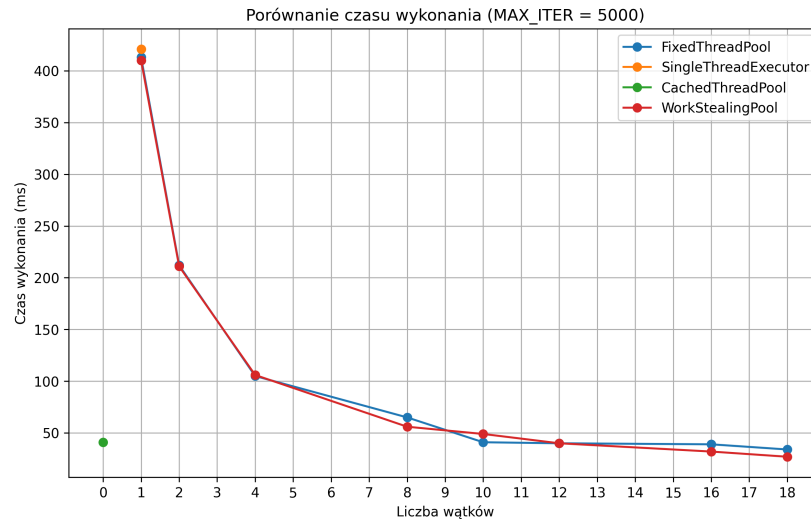
Wraz ze wzrostem iteracji przewaga puli dynamicznej (CachedThreadPool) zmniejsza się, a WorkStealingPool i FixedThreadPool zaczynają dominować. Optymalna liczba wątków dla większości implementacji wydaje się oscylować wokół liczby dostępnych rdzeni procesora.



Rysunek 3: Wykres czasu wykonania obliczeń w zależności od liczby wątków i użytego Executora dla $MAX_ITER = 2000$

4.4 Wyniki dla $MAX_ITER = 5000$

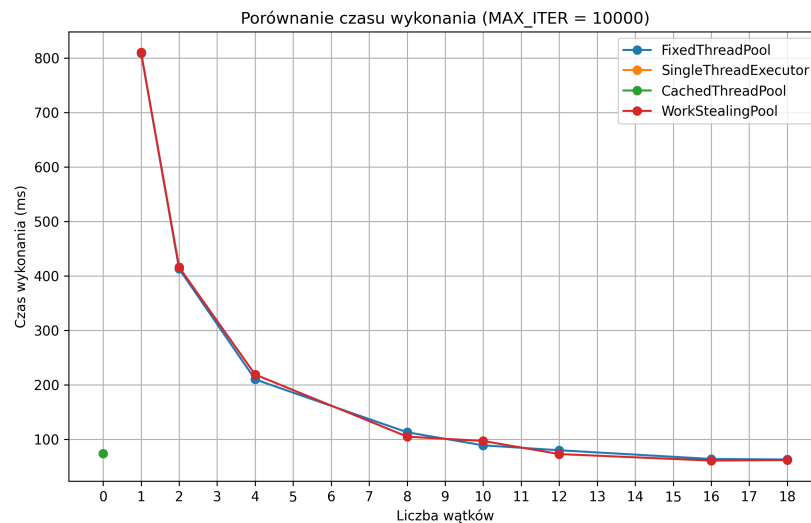
WorkStealingPool wykazuje najkrótszy czas dla większej liczby wątków, co potwierdza jego zdolność do dynamicznego rozkładu obciążenia. FixedThreadPool osiąga zbliżoną wydajność, ale wymaga bardziej ręcznego doboru liczby wątków. SingleThreadExecutor nadal jest najmniej efektywny, co pokazuje ograniczenia podejścia jednordzeniowego przy obliczeniach równoległych.



Rysunek 4: Wykres czasu wykonania obliczeń w zależności od liczby wątków i użytego Executora dla $MAX_ITER = 5000$

4.5 Wyniki dla $MAX_ITER = 10000$

Przy dużej liczbie iteracji różnice między WorkStealingPool a FixedThreadPool praktycznie znikają, ponieważ obie implementacje efektywnie rozkładają zadania. Wydajność CachedThreadPool jest stabilna, ale nie ma wyraźnej przewagi nad innymi implementacjami przy dużym obciążeniu. SingleThreadExecutor pozostaje bardzo wolny, co czyni go nieodpowiednim dla takich zadań.



Rysunek 5: Wykres czasu wykonania obliczeń w zależności od liczby wątków i użytego Executora dla $MAX_ITER = 10000$

4.6 Podsumowanie

Zastosowanie wzorców Executor i Future w implementacji programu obliczającego zbiór Mandelbrota pozwoliło na efektywne zarządzanie wielowątkowością i dynamiczne dostosowanie liczby aktywnych wątków do wymagań obliczeniowych. Dzięki temu możliwe było równoległe przetwarzanie danych, co znacząco skróciło czas wykonania programu w porównaniu do podejścia sekwencyjnego. Kluczowym elementem była elastyczność wykorzystania różnych implementacji Executor, takich jak FixedThreadPool, CachedThreadPool oraz WorkStealingPool, co pozwoliło na zbadanie ich wydajności w różnych konfiguracjach.

WorkStealingPool wyróżnił się zdolnością do dynamicznego balansowania obciążenia między wątkami, co zapewniło najwyższą wydajność przy dużej liczbie iteracji i nierównomiernym rozkładzie zadań. FixedThreadPool okazał się równie skuteczny, ale wymagał ręcznego doboru liczby wątków, co może być bardziej skomplikowane w scenariuszach o zmiennym obciążeniu. CachedThreadPool najlepiej sprawdził się w lekkich i krótkotrwałych zadaniach, jednak jego przewaga malała wraz ze wzrostem liczby iteracji i złożoności obliczeń. SingleThreadExecutor był najwolniejszy, co potwierdziło jego ograniczoną przydatność w zadaniach wymagających równoległego przetwarzania.

Zastosowana architektura pozwoliła na skuteczne skalowanie obliczeń w zależności od liczby wątków i specyfiki sprzętu, co przełożyło się na pełne wykorzystanie dostępnych zasobów procesora. Wprowadzenie mechanizmów równoległych przyczyniło się do znacznego przyspieszenia obliczeń, a jednocześnie zapewniło przewidywalność i stabilność działania programu w różnych konfiguracjach. Dzięki temu rozwiązanie jest zarówno elastyczne, jak i skalowalne, sprawdzając się w szerokim zakresie obciążeń.

Literatura

- [1] <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Future.html>
- [2] <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Executors.html>
- [3] <https://docs.oracle.com/javase/tutorial/essential/concurrency/executors.html>
- [4] <https://stackoverflow.com/questions/25169635/usage-of-generic-in-callable-and-future-interface>